Assignment 2:
Data Analysis Report

36103: Statistical Thinking for Data Science

Submitted: May 10, 2026

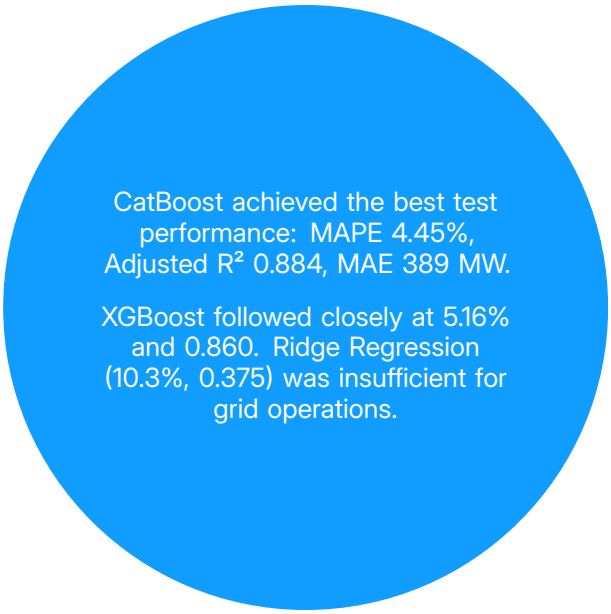
Group Number: 5

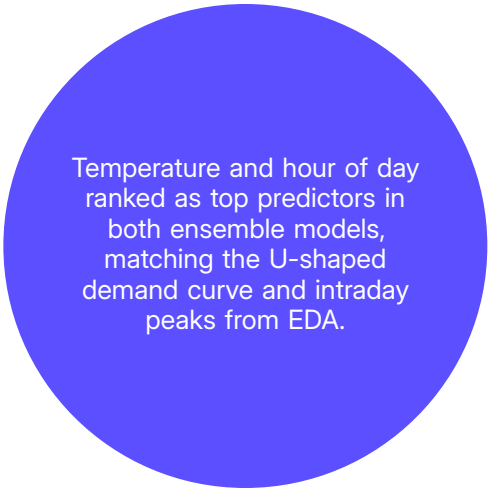
Yash Bankar	(25717428)
Preran Apinakoppa	(26583524)
Bhavya Subhash Chandar	(25599679)
Rohit Khanna	(25669108)
Murphy Fang	(26211831)

Executive Summary

Electricity demand in NSW varies with weather, time, and season. During peak periods, wholesale prices can reach $17,500/MWh, making accurate forecasts important for grid stability and cost management.

Analysing NSW demand (2022-2026), this study developed a 30-minute interval forecasting pipeline. CatBoost emerged as the optimal model, effectively capturing non-linear climatic and temporal drivers to support grid stability and market pricing strategies.





Contents

List of Tables

List of Figures

Introduction

1 Introduction

1.1 Problem Statement

Electricity supply and demand must balance every 30 minutes. When they don't, grid operators absorb the cost. NSW wholesale prices can reach $17,500/MWh under the market cap that applied through mid-2025 (AER, 2025b).

1.2 Rationale

Serving 11 million customers, the NEM relies on NSW as its primary generator (AER, 2026). NSW's 7.5 GW of rooftop solar (the highest globally per capita) creates a "duck curve" effect: suppressing midday demand before requiring a massive evening ramp from dispatchable generators (CEC, 2025; Green.com.au, 2026). This volatility drove 2024 wholesale prices up 43% to $150/MWh, making accurate forecasting critical to reducing costs(AER, 2025a).

1.3 Project Aims and Objectives

The aim of this project was to determine whether machine learning models trained on historical half-hourly NSW data can predict grid electricity demand accurately enough to support operational dispatch planning through three research questions:

1. Does temperature have a non-linear relationship with NSW grid demand across 2°C bins from 5°C to 43°C?
2. Do mean hourly demand profiles differ between weekdays, weekends, and public holidays in NSW?
3. Can a regression model predict 30-minute NSW grid demand with a MAPE below 5% on test data?

To address these, we merged AEMO market data with environmental and economic indicators. We compared Ridge (baseline), XGBoost, and CatBoost models using a 60:20:20 chronological split, evaluated via RMSE, MAE, MAPE, and Adjusted R^2.

Methodology

2 Methodology

2.1 Dataset

2.1.1 Data Acquisition and Cleaning

While raw AEMO market data was high-frequency, it lacked environmental and economic features needed for predictive modelling, which were synthetically merged using time-stamps as primary keys. Cleaning involved linear interpolation of missing observations, removing high multi-collinear features and frequency alignments. Please refer to Appendix Section A for a comprehensive **EDA**.

Table 1. Data Collation and Source Rationale

Data Source	Description	Methodology Rationale
AEMO (Market Data)	Half-hourly TOTAL_DEMAND and RRP (Regional Reference Price) for the NSW1 region.	Established the target variable and allowed for the analysis of price-demand elasticity during peak events.
AEMO (Solar Data)	Estimated ROOFTOP_SOLAR_MW generation across NSW.	Critical for deriving GROSS_DEMAND_MW, accounting for "behind-the-meter" generation that masks true load.
Bureau of Meteorology	Half-hourly temperature, humidity, wind speed, and cloud cover data for Sydney.	Included to model the primary driver of demand: the non-linear relationship between ambient temperature and HVAC load.
Economic Indicators	Monthly Cash Rate (RBA), Household Spend (ABS), and Consumer Confidence indices (ANZ-Roy Morgan Australian).	Integrated to capture macro-trends and shifts in baseline energy consumption patterns over the 4-year study period.

2.1.2 Feature Scaling

Feature engineering generated cyclical temporal encodings (Sine/Cosine transforms) to capture seasonal and daily periodicity. Data was standardised via One-Hot Encoding for categorical variables and Standard Scaling for numerical features. Detailed visualisations are provided in Appendix Section A.

2.2 Technologies Used

Developed in a Python 3.11 virtual environment, BeautifulSoup for web scraping, Pandas for data-frames, CatBoost and XGBoost libraries, scikit-learn, and Vegas-Altair for visualisation. Seeding was set to 42 for consistency and reproducibility. Resulted in 70,176 rows across 28 features. Chronologically split in 60:20:20 provided 42105:14035:14036 rows for training, validation and testing respectively.

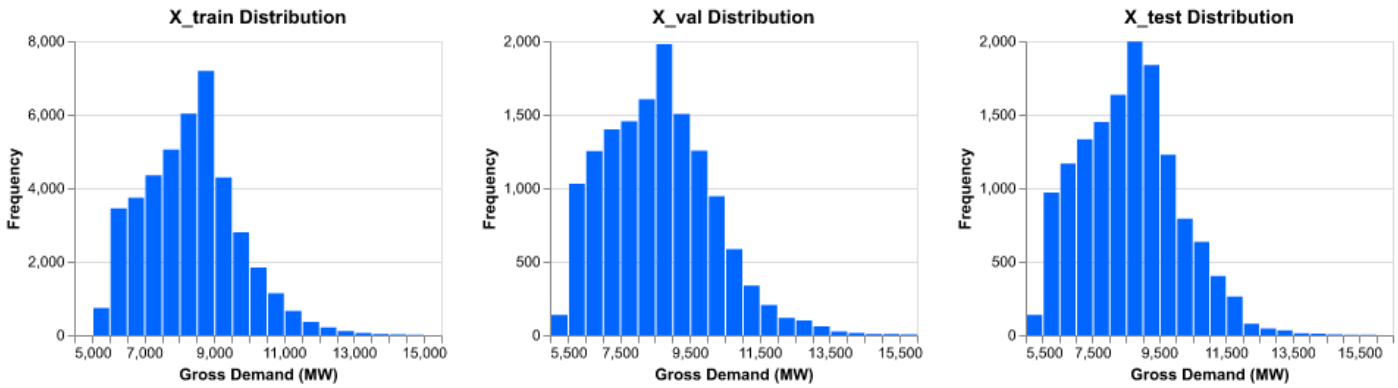


Figure 1. Target feature distribution

2.3 Models

Ridge Regression established a baseline using L2 regularisation to manage correlated weather predictors. XGBoost and CatBoost (GBDT) were implemented to capture non-linear relationships via sequential gradient descent. Hyperparameters were optimised via Grid Search with k-fold Cross-Validation.

Table 2. Model Selection and Theoretical Rationale

Model	Description	Methodology Rationale
Ridge	Linear model with L2 regularisation to penalise large coefficients.	Established a baseline to quantify non-linearity and manage multicollinearity among correlated weather predictors.
XGBoost	Scalable GBDT library designed for high computational efficiency.	Selected to address non-linear relationships in the data through sequential tree-building and dual regularisation (L1 and L2) to control model complexity.
CatBoost	Advanced GBDT algorithm that utilises symmetric trees and ordered boosting to reduce prediction shift and handle categorical features.	Chosen for its superior ability to generalise on unseen data and its efficient handling of categorical features (such as 'Season' and 'Holiday') without extensive manual encoding.

2.4 Grid Search

Hyperparameter optimisation is performed via Grid Search with k-fold Cross-Validation. Through exhaustive predefined parameter grids, we identified configurations that minimised generalisation error of each model.

2.5 Metrics

Table 3. Chosen Performance Metrics

Metric	Description and Purpose	Formula
MAE	Average absolute error in MW, operationally interpretable.	$\frac{1}{n}\sum\lvert y_i - \hat{y}_i\rvert$
MAPE	Scale-independent accuracy across demand magnitudes.	$\frac{1}{n}\sum\left\lvert\frac{y_i-\hat{y}_i}{y_i}\right\rvert \times 100$
RMSE	Penalises large outliers in peak demand periods.	$\sqrt{\frac{1}{n}\sum(y_i - \hat{y}_i)^2}$
Adjusted R^2	Explained variance penalised for predictor count.	$1 - \frac{(1-R^2)(n-1)}{n-p-1}$

Model performance was evaluated using a hold-out test set to ensure generalisability. The following Table 3 above illustrates the chosen metrics.

Results

3 Results

3.1 Key Findings

CatBoost achieved the best performance (MAPE 4.47%), significantly outperforming the Ridge baseline (MAPE 10.31%). This disparity confirms the fundamentally non-linear nature of NSW electricity demand.

3.2 In-depth Results

3.2.1 Can an ensemble regression model predict 30-minute NSW grid demand with MAPE below 5%?

Table 4. Model Performance Comparison

	RMSE (MW)			MAE (MW)			MAPE (%)			Adj R^2		
Model	**Train**	**Val**	**Test**	**Train**	**Val**	**Test**	**Train**	**Val**	**Test**	**Train**	**Val**	**Test**
Linear Regression	999.66	1202.67	1163.85	792.86	943.75	907.86	9.61	10.70	10.29	0.46	0.39	0.37
XGBoost	301.27	576.66	550.99	221.96	460.53	438.33	2.60	5.54	5.16	0.95	0.86	0.86
CatBoost	314.68	502.46	500.33	230.44	391.92	389.00	2.71	4.66	4.45	0.95	0.89	0.88

Three models were compared on held-out test data (Table 4). Ridge Regression failed to capture non-linear patterns (MAPE 10.31%, R^2 = 0.389). XGBoost improved but showed overfitting (MAPE 5.83%, R^2 = 0.836). CatBoost generalised best across all metrics, meeting the sub-5% threshold.

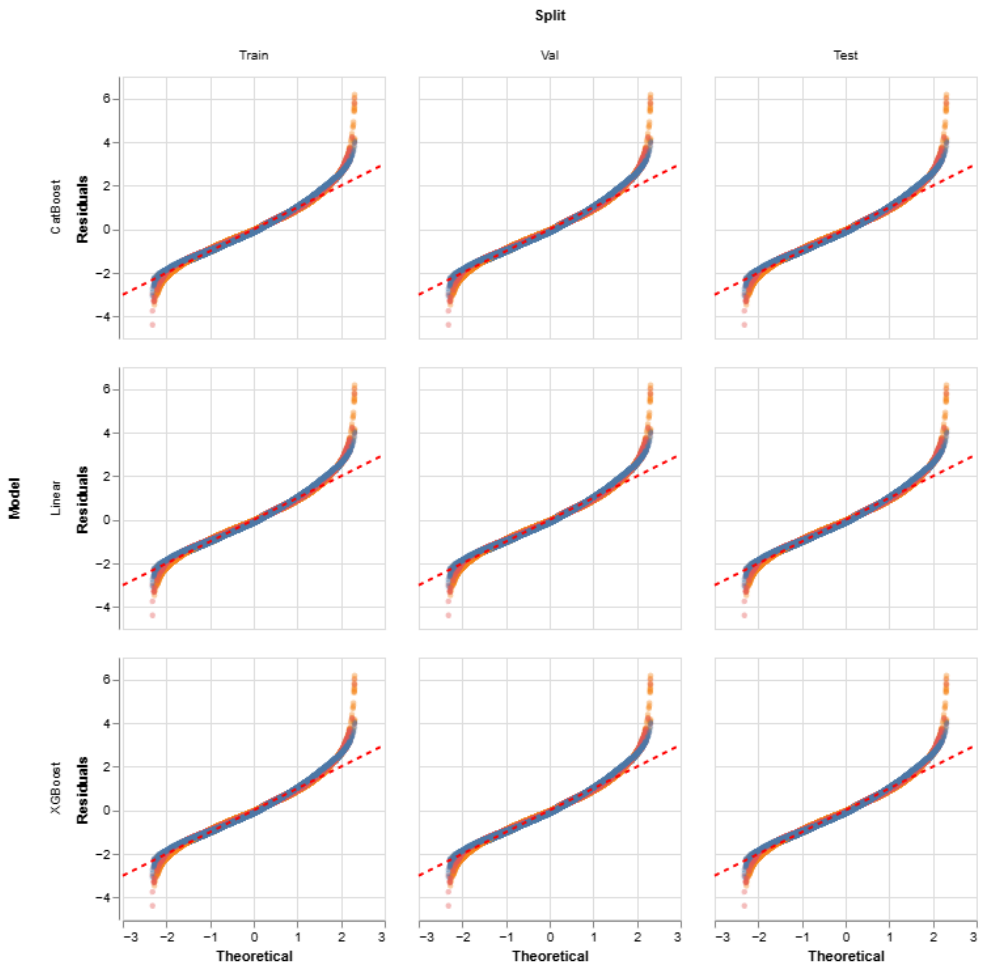
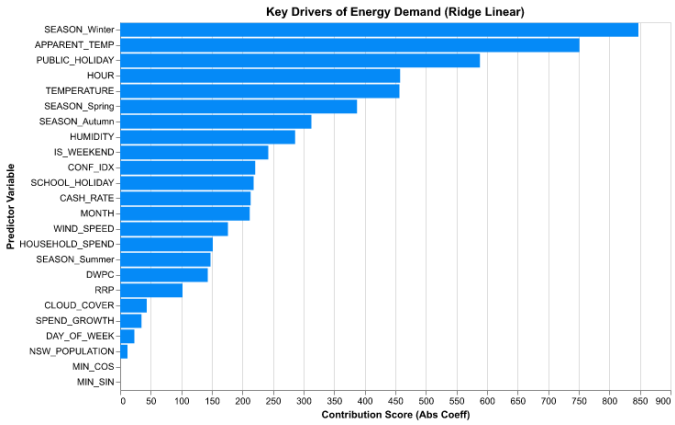


Figure 2. QQ-plots. All models show reasonable normality in the central quantile range with heavier tails at extremes, consistent with demand spikes during extreme weather events.

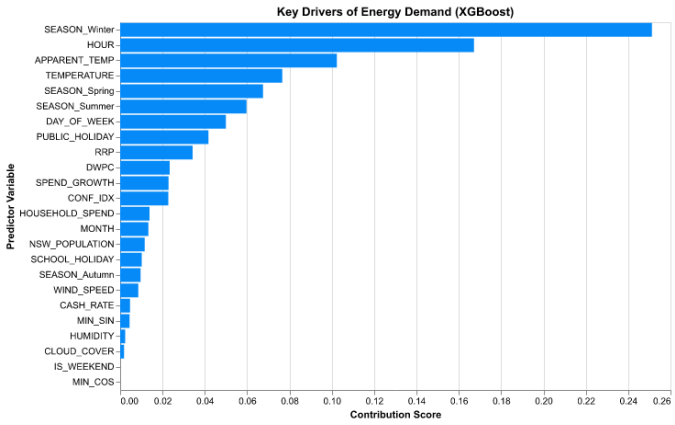
Regression assumptions were verified via QQ-plots (shown in Figure 2).

3.2.2 Model Interpretation and Feature Importance

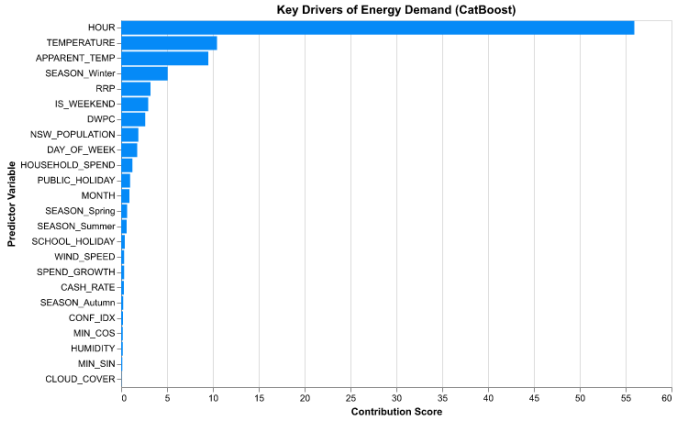
Feature importance comparisons (Figure 3) indicate that while the Ridge baseline relied on temperature, CatBoost and XGBoost prioritised temporal and environmental drivers. This confirms the boosted models learned solar patterns through proxies, successfully compensating for the removed rooftop solar feature.



(a) Ridge (Baseline)



(b) XGBoost



(c) CatBoost

Figure 3. Feature Importance Comparison across Models

Conversely, the marginal impact of the **Cash Rate** suggest that while economic indicators provide macro-context, they are less critical for short-term 30-minute interval forecasting than environmental and temporal features.

Conclusion

4 Conclusion

4.1 Conclusion

CatBoost achieved the sub-5% target (MAPE 4.47%), outperforming XGBoost and the Ridge baseline. This success confirms that gradient-boosted ensembles effectively capture the non-linear relationship between temperature, temporal cycles, and demand. However, with an MAE of $\sim$400$\sim$MW, the model remains a decision-support tool rather than a replacement for operator judgment, as errors of this magnitude represent significant financial risk at peak wholesale prices.

4.2 Project Limitations and Caveats

The model is fixed to a 2022-2026 training window with no self-update mechanism. Accuracy will drift as the grid changes. Urban areas also dominate the dataset, so regional and remote NSW performance is likely worse than the headline numbers suggest.

Remote and Aboriginal communities are underrepresented in AEMO grid data. A model trained mainly on urban demand risks reinforcing unequal energy distribution if applied broadly. Any deployment should respect Indigenous Data Sovereignty principles and involve community engagement before rollout.

4.3 Stakeholder Analysis and Project Outcomes

The deliverable is a CatBoost pipeline returning 30-minute demand forecasts from weather, calendar, and economic inputs. AEMO operators benefit most, in that the better forecasts mean less reliance on expensive last-minute generation. Next steps: live AEMO data integration, retraining cadence, and testing sequential models like LSTM for extreme weather periods.

References

References

Australian Energy Regulator. (2025a). *Wholesale markets quarterly - q4 2024* (tech. rep.) (Accessed: 2026-05-08). Australian Energy Regulator. https://www.aer.gov.au/publications/reports/performance/wholesale-markets-quarterly-q4-2024

Australian Energy Regulator. (2025b). *State of the energy market 2025* (tech. rep.) (Accessed: 2026-05-08). Australian Energy Regulator. https://www.aer.gov.au/publications/reports/performance/state-energy-market-2025

Australian Energy Regulator. (2026). Our role [Accessed: 2026-05-08].

Clean Energy Council. (2025). Australia powers ahead on rooftop solar as nation set to achieve 2030 rooftop target - new report finds [Accessed: 2026-05-08].

Green.com.au. (2026). Solar statistics 2026 [Accessed: 2026-05-08].

Appendices

Appendices

A Exploratory Data Analysis

The objective of this exploratory phase was to move beyond the raw complexity to identify the primary drivers of Gross Energy Demand. In light of the data science project cycle, this EDA served two critical functions: **Feature Selection** and **Operational Insights**.

A.1 General Exploration

Observing multicollinearity such as through a correlation heatmap is critical because it identifies highly redundant predictors that can destabilise the model, inflate the variance of coefficient estimates, and make it impossible to determine the individual mathematical contribution of each variable to the target output.

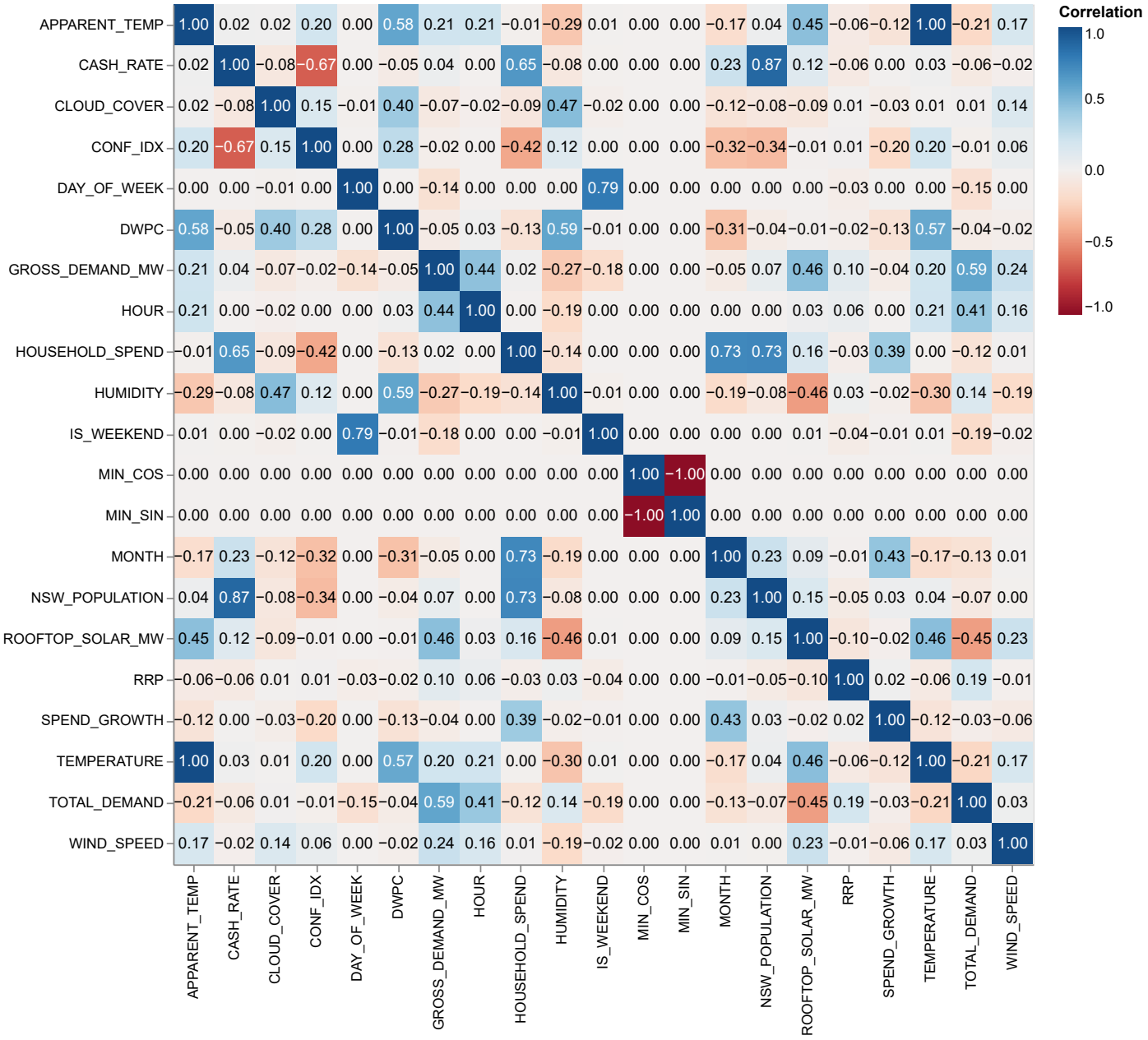


Figure 4. Multicollinearity Matrix on Raw Data

A.2 Does temperature have a non-linear relationship with NSW grid demand across 2°C bins?

A clear U-shaped relationship was observed between temperature and mean grid demand (Figure 5). Demand was lowest near 21°C and rose sharply at both cold and hot extremes, directly ruling out linear modelling.

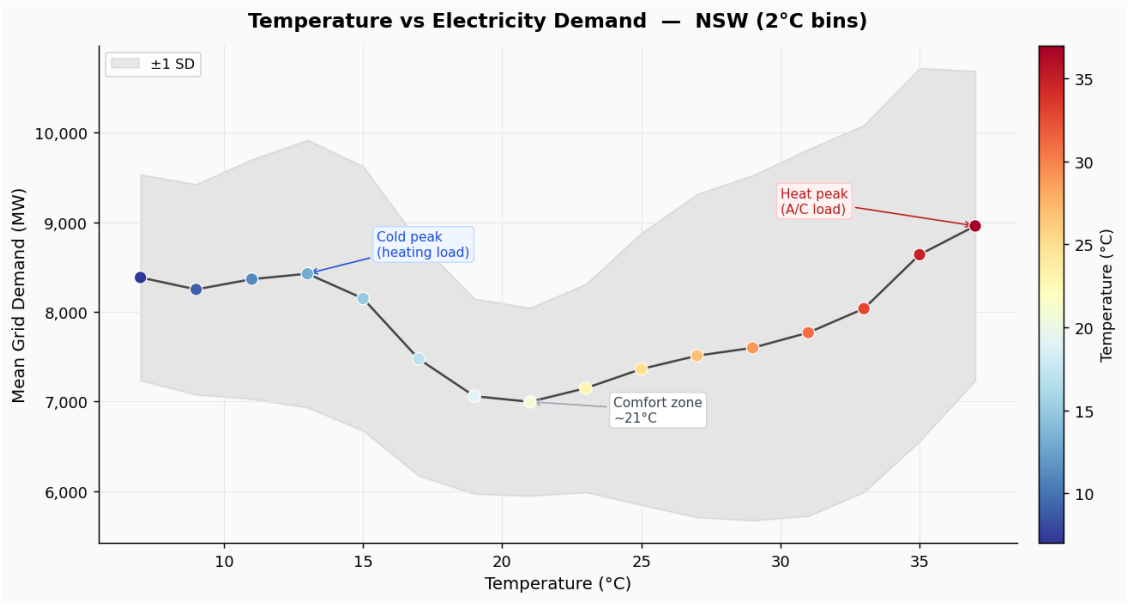


Figure 5. Mean NSW grid electricity demand (MW) across 2°C temperature bins (5°C-43°C). A U-shaped pattern is evident, with lowest demand near the comfort zone (∼21°C, ∼7,000 MW), rising at cold extremes due to heating load and hot extremes due to air-conditioning. Shaded band represents ±1 SD.

A.3 Do mean hourly demand profiles differ between weekdays, weekends, and public holidays?

Weekday demand showed a morning commercial peak at 8 AM (∼8,200 MW) and an evening residential peak at 7 PM (∼9,500 MW). Weekend and holiday profiles were 1,000-1,500 MW lower with no morning peak (Figure 6). An unexpected finding was a pronounced midday dip across all day types driven by rooftop solar, forming the classic "duck curve" pattern (Figure 7).

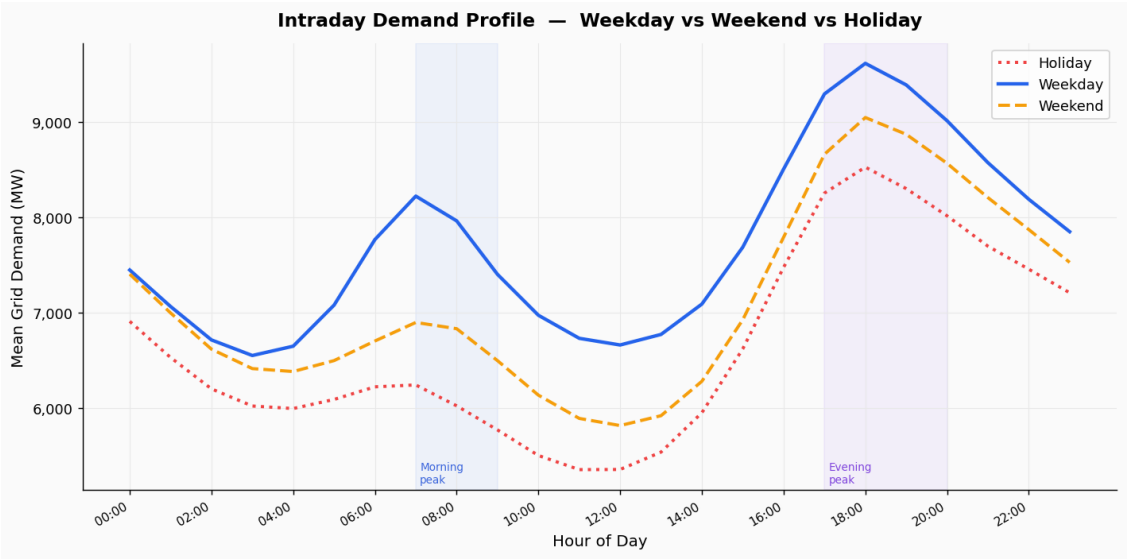


Figure 6. Mean NSW grid electricity demand (MW) by hour of day for weekdays (solid blue), weekends (dashed orange), and public holidays (dotted red). Weekdays show a morning peak at 8 AM (∼8,200 MW) and an evening peak at 7 PM (∼9,500 MW). Weekend and holiday profiles are 1,000-1,500 MW lower with no commercial morning peak.

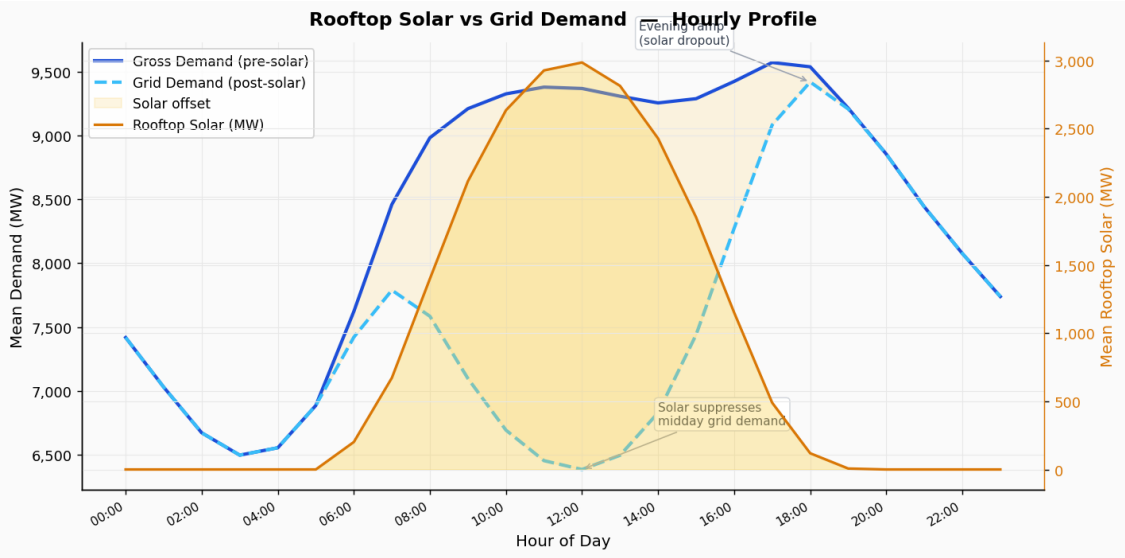


Figure 7. Mean hourly rooftop solar generation (MW, right axis, orange) overlaid on gross pre-solar demand (solid blue) and net post-solar grid demand (dashed teal). Solar peaks near midday (∼3,000 MW), suppressing grid demand and creating the sharp evening ramp as generation drops — the "duck curve" pattern.

B Modelling Code

B.1 Data Preparation

```
import pandas as pd
import os

## IMPORTING THE AEMO DATA

# relative directory path
directory = './data/demand'
dfs = []

for filename in os.listdir(directory):
    if filename.endswith('.csv'):
        # Create the full file path and read the file
        file_path = os.path.join(directory, filename)
        df_temp = pd.read_csv(file_path)

        # FIX THE FIRST ROW IN THE LOOP
        df_temp['SETTLEMENTDATE'] = pd.to_datetime(df_temp['SETTLEMENTDATE'])
        if not df_temp.empty:
            df_temp.loc[0, 'SETTLEMENTDATE'] -= pd.Timedelta(minutes=5)

        # Check if column exists to prevent errors
        print(f"File: {filename}")
        dfs.append(df_temp)

df = pd.concat(dfs, ignore_index=True)
df.drop(columns=['REGION', 'PERIODTYPE'], inplace=True)
df.rename(columns={'SETTLEMENTDATE': 'DATE_TIME', 'TOTALDEMAND': 'TOTAL_DEMAND'},
    ↪ inplace=True, errors='ignore')

df['DATE_TIME'] = pd.to_datetime(df['DATE_TIME'])
df = df[df['DATE_TIME'].dt.minute.isin([0, 30])].copy()
df = df.sort_values('DATE_TIME').reset_index(drop=True)

## CREATING COLUMNS FOR SCHOOL_HOLIDAY, PUBLIC_HOLIDAY
import holidays

# Convert to Pandas datetime for better parsing
df['DATE_TIME'] = pd.to_datetime(df['DATE_TIME'])

nsw_holidays = holidays.Australia(subdiv='NSW', years=range(2021, 2027))
df['PUBLIC_HOLIDAY'] = df['DATE_TIME'].dt.date.isin(nsw_holidays)

school_holiday_ranges = [
    # 2022
    ('2022-01-01', '2022-01-27'), # Summer 21-22 end
    ('2022-04-11', '2022-04-22'), # Autumn
    ('2022-07-04', '2022-07-15'), # Winter
    ('2022-09-26', '2022-10-07'), # Spring
    ('2022-12-21', '2022-12-31'), # Summer 22-23 start

    # 2023
    ('2023-01-01', '2023-01-26'), # Summer 22-23
    ('2023-04-10', '2023-04-21'), # Autumn
```

```
    ('2023-07-03', '2023-07-14'), # Winter
    ('2023-09-25', '2023-10-06'), # Spring
    ('2023-12-20', '2023-12-31'), # Summer 23-24 start

    # 2024
    ('2024-01-01', '2024-01-29'), # Summer 23-24 end
    ('2024-04-15', '2024-04-26'), # Autumn
    ('2024-07-08', '2024-07-19'), # Winter
    ('2024-09-30', '2024-10-11'), # Spring
    ('2023-12-23', '2023-12-31'), # Summer 24-25 start

    # 2025
    ('2025-01-01', '2025-01-30'), # Summer 24-25 end
    ('2025-04-14', '2025-04-24'), # Autumn
    ('2025-07-07', '2025-07-18'), # Winter
    ('2025-09-29', '2025-10-10'), # Spring
    ('2025-12-22', '2025-12-31'), # Summer 25-26 start

    # 2026
    ('2026-01-01', '2026-01-26'), # Summer 25-26 end
    ('2026-04-07', '2026-04-17'), # Autumn
    ('2026-07-06', '2026-07-17'), # Winter
    ('2026-09-28', '2026-10-09'), # Spring
    ('2026-12-18', '2026-12-31'), # Summer 26-27 start
]

holiday_dates = set()
for start, end in school_holiday_ranges:
    holiday_dates.update(pd.date_range(start, end).date)

df['SCHOOL_HOLIDAY'] = df['DATE_TIME'].dt.date.isin(holiday_dates) # check and store
    ↪ boolean


## WEATHER DATA
import pandas as pd
import datetime

def fetch_sydney_weather(start_year, end_year):
    # Sydney Airport Station ID is usually 'YSSY' in the ASOS network
    station = 'YSSY'
    start_date = f"{start_year}-01-01+00:00"
    end_date = f"{end_year}-12-31+23:59"

    # Web scraping the global archive - added 'feel' and 'p01m' for transpiration
        ↪ context
    url = (f"https://mesonet.agron.iastate.edu/cgi-bin/request/asos.py?"
           f"station={station}&data=tmpc&data=dwpc&data=relh&data=feel&data=sknt&"
           f"data=skyc1&data=skyc2&data=skyc3&"
           f"year1={start_year}&month1=1&day1=1&"
           f"year2={end_year}&month2=12&day2=31&"
           f"tz=Etc%2FGMT-10&format=onlycomma&latlon=no&missing=M")

    print("Fetching weather data")
    weather_df = pd.read_csv(url, skiprows=0, na_values='M')

```

```
    # Rename columns for clarity
    weather_df = weather_df.rename(columns={
        'valid': 'DATE_TIME',
        'tmpc': 'TEMPERATURE',
        'relh': 'HUMIDITY',
        'dwpc': 'DWPC',
        'feel': 'APPARENT_TEMP',
        'sknt': 'WIND_SPEED',
        'skyc1': 'CLOUD_L1',
        'skyc2': 'CLOUD_L2',
        'skyc3': 'CLOUD_L3'
    })

    weather_df['DATE_TIME'] = pd.to_datetime(weather_df['DATE_TIME'])
    weather_df['APPARENT_TEMP'] = ((weather_df['APPARENT_TEMP'] - 32) * 5/9) # Farenheit to Celcius
    weather_df['WIND_SPEED'] = (weather_df['WIND_SPEED'] * 1.852) # Convert Wind Speed from knots to km/h (1 knot  1.852 km/h)

    return weather_df

weather_df = fetch_sydney_weather(2022, 2026)

# Cleaning dataset
cloud_map = {'CLR': 0, 'SKC': 0, 'FEW': 2, 'SCT': 4, 'BKN': 6, 'OVC': 8}
for col in ['CLOUD_L1', 'CLOUD_L2', 'CLOUD_L3']:
    weather_df[col] = weather_df[col].map(cloud_map).fillna(0).round()

weather_df['CLOUD_COVER'] = weather_df[['CLOUD_L1', 'CLOUD_L2', 'CLOUD_L3']].max(axis=1)

weather_df.drop(['station', 'CLOUD_L1', 'CLOUD_L2', 'CLOUD_L3'], axis=1, inplace=True, errors='ignore')

if 'DATE_TIME' in df.columns:
    demand_df = df.set_index('DATE_TIME').sort_index()
else:
    demand_df = df.sort_index()

if 'DATE_TIME' in weather_df.columns:
    weather_df = weather_df.set_index('DATE_TIME').sort_index()
else:
    weather_df = weather_df.sort_index()

weather_numeric = weather_df.select_dtypes(include=['number', 'float', 'int'])
weather_5min = weather_numeric.reindex(demand_df.index)

# Interpolate the gaps (Linear interpolation)
weather_5min = weather_5min.interpolate(method='linear')
weather_5min = weather_5min.bfill().ffill()

# merge back together
demand_df = demand_df.drop(columns=weather_5min.columns.intersection(demand_df.columns))
df = pd.concat([demand_df, weather_5min], axis=1).reset_index()

```

```
if 'index' in df.columns:
    df = df.rename(columns={'index': 'DATE_TIME'})


### Consumer spending and confidence


## RBA Monthly Cash Rate
cash_rate_df = pd.read_csv('./data/rba_monthly_cash_rate.csv')
df['Month_Year'] = df['DATE_TIME'].dt.strftime('%Y-%m') # parse date format like '2023-01'
df = df.merge(cash_rate_df, left_on='Month_Year', right_on='DATE', how='left') # broadcast join
df = df.drop(columns=['Month_Year', 'DATE']) # clean columns

## Monthly Household Spending Indicator (MHSI)
spending_df = pd.read_csv('./data/spending_millions.csv')
df['join_key'] = df['DATE_TIME'].dt.strftime('%b-%y')

# Merge and clean up
df = df.merge(spending_df, left_on='join_key', right_on='Month', how='left')
df = df.drop(columns=['join_key', 'Month'])

# Consumer Confidence (Westpac)
consumer_conf = pd.read_csv('./data/consumer_conf.csv')
df['month_key'] = df['DATE_TIME'].dt.strftime('%Y-%m')
df = df.merge(consumer_conf, left_on='month_key', right_on='year-month', how='left')
df = df.drop(columns=['month_key', 'year-month'])

pop_df = pd.read_csv('./data/population.csv')
pop_df['DATE_TIME'] = pd.to_datetime(pop_df['DATE_TIME'])

df = pd.merge(df, pop_df, on='DATE_TIME', how='left')
df['NSW_POPULATION'] = df['NSW_POPULATION'].interpolate(method='linear').round().astype(int)

from nemosis import dynamic_data_compiler
import pandas as pd

# Define project range
start_time = '2022/01/01 00:00:00'
end_time = '2026/02/01 00:00:00'

solar_df = dynamic_data_compiler(
    start_time=start_time,
    end_time=end_time,
    table_name='ROOFTOP_PV_ACTUAL',
    raw_data_location='./data/cache'  # This is the required positional argument
)

# Filter for NSW and clean up
solar_nsw = solar_df[solar_df['REGIONID'] == 'NSW1'].copy()

# AEMO ROOFTOP_PV_ACTUAL usually uses 'INTERVAL_DATETIME' or 'SETTLEMENTDATE'
date_col = 'INTERVAL_DATETIME' if 'INTERVAL_DATETIME' in solar_nsw.columns else 'SETTLEMENTDATE'
```

```

solar_nsw = solar_nsw[[date_col, 'POWER']].rename(
    columns={date_col: 'DATE_TIME', 'POWER': 'ROOFTOP_SOLAR_MW'}
)

solar_nsw['DATE_TIME'] = pd.to_datetime(solar_nsw['DATE_TIME'])

# Aggregate duplicates to prevent row-doubling
solar_nsw = solar_nsw.groupby('DATE_TIME')['ROOFTOP_SOLAR_MW'].mean().reset_index()

# Perform the merge
df = pd.merge(df, solar_nsw[['DATE_TIME', 'ROOFTOP_SOLAR_MW']], on='DATE_TIME', how=
    ↪ 'left')
df['ROOFTOP_SOLAR_MW'] = df['ROOFTOP_SOLAR_MW'].bfill(limit=1)

# Standard linear interpolation for any other gaps
df['ROOFTOP_SOLAR_MW'] = df['ROOFTOP_SOLAR_MW'].interpolate(method='linear')

# Zero out nighttime values
df.loc[(df['DATE_TIME'].dt.hour >= 20) | (df['DATE_TIME'].dt.hour <= 5), '
    ↪ ROOFTOP_SOLAR_MW'] = 0
df['ROOFTOP_SOLAR_MW'] = df['ROOFTOP_SOLAR_MW'].fillna(0)

df['GROSS_DEMAND_MW'] = df['TOTAL_DEMAND'] + df['ROOFTOP_SOLAR_MW']

# round and save
df = df.round(4)
df.to_csv('./data/dataset.csv', index=False)
```

B.2 Modelling

```
# # Preprocessing

! pip install -r requirements.txt -q


import pandas as pd
import numpy as np

df = pd.read_csv('./data/dataset.csv')

df.head()

print(df.shape)
df.info()
df.describe()

df.head()

# ## Feature Engineering

df['DATE_TIME'] = pd.to_datetime(df['DATE_TIME'])

df['HOUR'] = df['DATE_TIME'].dt.hour
df['DAY_OF_WEEK'] = df['DATE_TIME'].dt.dayofweek
df['MIN_SIN'] = np.sin(2 * np.pi * df['DATE_TIME'].dt.minute / 60)
df['MIN_COS'] = np.cos(2 * np.pi * df['DATE_TIME'].dt.minute / 60)
```

```
df['IS_WEEKEND'] = df['DAY_OF_WEEK'].isin([5, 6]).astype(int)
df['MONTH'] = df['DATE_TIME'].dt.month

def get_season(month):
    if month in [12, 1, 2]:
        return 'Summer'
    elif month in [3, 4, 5]:
        return 'Autumn'
    elif month in [6, 7, 8]:
        return 'Winter'
    else:
        return 'Spring'

df['SEASON'] = df['MONTH'].apply(get_season)
df = pd.get_dummies(df, columns=['SEASON'], prefix='SEASON')

df.shape

import altair as alt
import pandas as pd

# Calculate the correlation matrix for numerical columns
corr_matrix = df.select_dtypes('number').corr().reset_index().melt(id_vars='index')
corr_matrix.columns = ['Variable 1', 'Variable 2', 'Correlation']

# Create the heatmap
base = alt.Chart(corr_matrix).encode(
    x=alt.X('Variable 1:O', title=None),
    y=alt.Y('Variable 2:O', title=None)
)

heatmap = base.mark_rect().encode(
    color=alt.Color('Correlation:Q', scale=alt.Scale(scheme='redblue', domain=[-1,
        ↪ 1]))
)

# Add text labels for the correlation coefficients
text = base.mark_text().encode(
    text=alt.Text('Correlation:Q', format='.2f'),
    color=alt.condition(
        "abs(datum.Correlation) > 0.5",  # Using a string expression instead
        alt.value('white'),
        alt.value('black')
    )
)

chart = (heatmap + text).properties(
    width=600,
    height=600
)

chart.save('./figs/multicollinearity_chart.pdf')

display(chart)

## Splitting the Dataset
```

```

from sklearn.model_selection import train_test_split


y_target = 'GROSS_DEMAND_MW'  # set the target variable
# y_target = 'TOTAL_DEMAND'  # set the target variable
# y_target = 'ROOFTOP_SOLAR_MW'  # set the target variable

df = df.sort_values('DATE_TIME')
df.drop(columns=[
    'TOTAL_DEMAND',
    'ROOFTOP_SOLAR_MW'
], inplace=True, errors='ignore')
df.drop(columns=['DATE_TIME'], inplace=True, errors='ignore')

n = len(df)
train_end = int(n * 0.6)
val_end = int(n * 0.8)

# X_train, X_temp = train_test_split(df, test_size=0.4, random_state=42)
# X_val, X_test = train_test_split(X_temp, test_size=0.5, random_state=42)

X_train = df.iloc[:train_end]
X_val = df.iloc[train_end:val_end]
X_test = df.iloc[val_end:]

def plot_demand_dist(df, title):
    """
    Generates a binned distribution chart for GROSS_DEMAND_MW.
    """
    return alt.Chart(df).mark_bar().encode(
        alt.X("GROSS_DEMAND_MW:Q", bin=alt.Bin(maxbins=30), title="Gross Demand (MW)
            ↪ "),
        alt.Y("count()", title="Frequency"),
        tooltip=["count()"],
        color=alt.value('#0066ff')
    ).properties(
        width=250,
        height=200,
        title=title
    )

# Execute for all datasets
chart_train = plot_demand_dist(X_train, "X_train Distribution")
chart_val = plot_demand_dist(X_val, "X_val Distribution")
chart_test = plot_demand_dist(X_test, "X_test Distribution")

# Display side-by-side
final_dist_chart = (chart_train | chart_val | chart_test)
final_dist_chart.save('./figs/dist_chart.png')
display(final_dist_chart)

y_train, y_val, y_test = (
    X_train.pop(y_target),
    X_val.pop(y_target),
    X_test.pop(y_target),
```

```
)

df_names = ['X_train', 'X_val', 'X_test']


print(X_train.shape)
print(X_val.shape)
print(X_test.shape)


# ## Data Transformation


# ### One-hot Encoding (OHE)

from sklearn.preprocessing import OneHotEncoder

ohe_encoder = OneHotEncoder()

for name in df_names:
    df_ = globals()[name]

    globals()[name] = df_.copy()


# ### Feature Scaling

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
numeric_cols = X_train.select_dtypes(include=['number']).columns

scaler.fit(X_train[numeric_cols])

for name in df_names:
    df_ = globals()[name]

    df_[numeric_cols] = scaler.transform(df_[numeric_cols])

    for col in df_.columns:
        if df_[col].dtype == 'bool':
            df_[col] = df_[col].astype('category').cat.codes

    globals()[name] = df_.copy()

from sklearn.linear_model import Ridge

lin_reg = Ridge(
    alpha=1.0,
    fit_intercept=True,
    random_state=42
)

lin_reg.fit(X_train, y_train)

lin_train_preds = lin_reg.predict(X_train)
```

```
lin_val_preds = lin_reg.predict(X_val)
lin_test_preds = lin_reg.predict(X_test)

print(f"Model Intercept: {lin_reg.intercept_}")

# %%
from xgboost import XGBRegressor

xgb_model = XGBRegressor(
    n_estimators=1000,
    learning_rate=0.05,
    max_depth=6,
    random_state=42
)

xgb_model.fit(
    X_train, y_train,
    eval_set=[(X_val, y_val)],
    verbose=False
)

xgb_train_preds = xgb_model.predict(X_train)
xgb_val_preds = xgb_model.predict(X_val)
xgb_test_preds = xgb_model.predict(X_test)

# %%
from catboost import CatBoostRegressor

cat_model = CatBoostRegressor(
    iterations=1000,
    learning_rate=0.05,
    depth=6,
    loss_function='RMSE',
    random_state=42
)

cat_model.fit(
    X_train, y_train,
    eval_set=(X_val, y_val),
    early_stopping_rounds=50,
    verbose=False
)

cat_train_preds = cat_model.predict(X_train)
cat_val_preds = cat_model.predict(X_val)
cat_test_preds = cat_model.predict(X_test)

# %%
import pandas as pd
import numpy as np
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

def get_metrics_row(y_train, p_train, y_val, p_val, y_test, p_test, n_feat):
    """Calculate MultiIndex row data for model evaluation."""
    def score(y, p):
        r2 = r2_score(y, p)
```

```
        # Prevent division by zero if n_feat is too high relative to sample size
        adj_r2 = 1 - (1 - r2) * (len(y) - 1) / max((len(y) - n_feat - 1), 1)

        # Calculate MAPE with a small epsilon to avoid division by zero
        mape = np.mean(np.abs((y - p) / np.maximum(np.abs(y), 1e-10))) * 100

        return {
            'RMSE (MW)': round(np.sqrt(mean_squared_error(y, p)), 6),
            'MAE (MW)': round(mean_absolute_error(y, p), 6),
            'MAPE (%)': round(mape, 6),
            'Adj R2': round(adj_r2, 6)
        }

    tr, vl, ts = score(y_train, p_train), score(y_val, p_val), score(y_test, p_test)

    return {
        ('RMSE (MW)', 'Train'): tr['RMSE (MW)'], ('RMSE (MW)', 'Val'): vl['RMSE (MW)
            ↪ '], ('RMSE (MW)', 'Test'): ts['RMSE (MW)'],
        ('MAE (MW)', 'Train'): tr['MAE (MW)'],   ('MAE (MW)', 'Val'): vl['MAE (MW)'
            ↪ ],   ('MAE (MW)', 'Test'): ts['MAE (MW)'],
        ('MAPE (%)', 'Train'): tr['MAPE (%)'],   ('MAPE (%)', 'Val'): vl['MAPE (%)'
            ↪ ],   ('MAPE (%)', 'Test'): ts['MAPE (%)'],
        ('Adj R2', 'Train'): tr['Adj R2'],       ('Adj R2', 'Val'): vl['Adj R2'],
            ↪       ('Adj R2', 'Test'): ts['Adj R2']
    }

n_feat = X_train.shape[1]

# Data aggregation - Added n_feat to each call
data = {
    'Linear Regression': get_metrics_row(y_train, lin_train_preds, y_val,
        ↪ lin_val_preds, y_test, lin_test_preds, n_feat),
    'XGBoost': get_metrics_row(y_train, xgb_train_preds, y_val, xgb_val_preds,
        ↪ y_test, xgb_test_preds, n_feat),
    'CatBoost': get_metrics_row(y_train, cat_train_preds, y_val, cat_val_preds,
        ↪ y_test, cat_test_preds, n_feat)
}

# DataFrame construction
metrics_df = pd.DataFrame.from_dict(data, orient='index')
metrics_df.columns = pd.MultiIndex.from_tuples(metrics_df.columns)

# Styling for display
styled_df = metrics_df.style.set_table_styles([
    {'selector': 'th', 'props': [('text-align', 'center'), ('background-color', "
        ↪ #0077FF"), ('color', 'white')]},
    {'selector': 'td', 'props': [('text-align', 'center')]}
]).set_properties(**{'text-align': 'center'})

display(styled_df)

# %%
from sklearn.linear_model import Ridge
from sklearn.model_selection import GridSearchCV

# Define parameter grid for Linear Model (Ridge)
```

```
# alpha is the regularization strength
linear_param_grid = {
    'alpha': [0.1, 1.0, 10.0, 100.0],
    'fit_intercept': [True, False]
}

# Create GridSearchCV for Linear Model
linear_grid_search = GridSearchCV(
    Ridge(),
    linear_param_grid,
    cv=3,
    scoring='neg_mean_squared_error',
    n_jobs=-1
)

# Fit the grid search
linear_grid_search.fit(X_train, y_train)

# Get best model and parameters
best_linear_model = linear_grid_search.best_estimator_
print(f"Best Linear parameters: {linear_grid_search.best_params_}")
print(f"Best CV score (neg_mse): {linear_grid_search.best_score_}")

# Make predictions with best model
linear_grid_train_preds = best_linear_model.predict(X_train)
linear_grid_val_preds = best_linear_model.predict(X_val)
linear_grid_test_preds = best_linear_model.predict(X_test)

# %%
from sklearn.model_selection import GridSearchCV

# Define parameter grid for CatBoost
param_grid = {
    'depth': [x for x in range(4, 9)],
    'learning_rate': [0.01, 0.05, 0.1],
    'iterations': [500, 1000, 1500]
}

# Create GridSearchCV for CatBoost
cat_grid_search = GridSearchCV(
    CatBoostRegressor(loss_function='RMSE', random_state=42, verbose=False),
    param_grid,
    cv=3,
    scoring='neg_mean_squared_error',
    n_jobs=-1
)

# Fit the grid search
cat_grid_search.fit(X_train, y_train)

# Get best model and parameters
best_cat_model = cat_grid_search.best_estimator_
print(f"Best CatBoost parameters: {cat_grid_search.best_params_}")
print(f"Best CV score (neg_mse): {cat_grid_search.best_score_}")

# Make predictions with best model
```

```
cat_grid_train_preds = best_cat_model.predict(X_train)
cat_grid_val_preds = best_cat_model.predict(X_val)
cat_grid_test_preds = best_cat_model.predict(X_test)

from sklearn.model_selection import GridSearchCV

# Define parameter grid for XGBoost
xgb_param_grid = {
    'max_depth': [x for x in range(4, 9)],
    'learning_rate': [0.01, 0.05, 0.1],
    'n_estimators': [500, 1000, 1500]
}

# Create GridSearchCV for XGBoost
xgb_grid_search = GridSearchCV(
    xgb_model.__class__(random_state=42),
    xgb_param_grid,
    cv=3,
    scoring='neg_mean_squared_error',
    n_jobs=-1
)

# Fit the grid search
xgb_grid_search.fit(X_train, y_train)

# Get best model and parameters
best_xgb_model = xgb_grid_search.best_estimator_
print(f"Best XGBoost parameters: {xgb_grid_search.best_params_}")
print(f"Best CV score (neg_mse): {xgb_grid_search.best_score_}")

# Make predictions with best model
xgb_grid_train_preds = best_xgb_model.predict(X_train)
xgb_grid_val_preds = best_xgb_model.predict(X_val)
xgb_grid_test_preds = best_xgb_model.predict(X_test)

import altair as alt
import pandas as pd
import numpy as np
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# Assuming X_train is the feature matrix
n_feat = X_train.shape[1]

# Data aggregation - Added n_feat to each call
data = {
    "Linear Regression": get_metrics_row(
        y_train,
        linear_grid_train_preds,
        y_val,
        linear_grid_val_preds,
        y_test,
        linear_grid_test_preds,
        n_feat,
    ),
    "XGBoost": get_metrics_row(
        y_train,
```

```
        xgb_grid_train_preds,
        y_val,
        xgb_grid_val_preds,
        y_test,
        xgb_grid_test_preds,
        n_feat,
    ),
    "CatBoost": get_metrics_row(
        y_train,
        cat_grid_train_preds,
        y_val,
        cat_grid_val_preds,
        y_test,
        cat_grid_test_preds,
        n_feat,
    ),
}

# DataFrame construction
metrics_df = pd.DataFrame.from_dict(data, orient="index")
metrics_df.columns = pd.MultiIndex.from_tuples(metrics_df.columns)

# Styling for display
styled_df = metrics_df.style.set_table_styles(
    [
        {
            "selector": "th",
            "props": [
                ("text-align", "center"),
                ("background-color", "#0077FF"),
                ("color", "white"),
            ],
        },
        {"selector": "td", "props": [("text-align", "center")]},
    ]
).set_properties(**{"text-align": "center"})

display(styled_df)

## Regression plots
alt.data_transformers.enable("vegafusion") # for large plots

plot_data = pd.DataFrame({
    'Actual': y_test.values,
    'Linear Regression': linear_grid_test_preds,
    'XGBoost': xgb_grid_test_preds,
    'CatBoost': cat_grid_test_preds
}).reset_index()

melted_df = plot_data.melt(id_vars=['index', 'Actual'],
                           var_name='Model',
                           value_name='Prediction')

actual_line = alt.Chart(melted_df).mark_line(color='black', strokeWidth=2, opacity
    ↪ =0.5).encode(
    x=alt.X('index:Q', title='Sample Index'),
```

```
    y=alt.Y('Actual:Q', title='MW'),
)

model_order = ['Linear Regression', 'XGBoost', 'CatBoost']

pred_line = alt.Chart(melted_df).mark_line(strokeDash=[5, 5]).encode(
    x='index:Q',
    y='Prediction:Q',
    color=alt.Color('Model:N',
                    scale=alt.Scale(scheme='tableau10'),
                    sort=model_order) # disable sorting
)

final_chart = alt.layer(actual_line, pred_line).properties(
    width=700,
    height=200
).facet(
    row=alt.Row('Model:N', sort=model_order) # disable sorting
).resolve_scale(
    y='shared'
)

display(final_chart)


import altair as alt
import pandas as pd
import numpy as np

def plot_feature_importance(model, model_name, columns, save_path=None):
    g_color = '#058af7'

    # Extract importance based on model type
    if hasattr(model, 'coef_'):
        importance = np.abs(model.coef_)
        x_title = 'Contribution Score (Abs Coeff)'
    elif hasattr(model, 'get_feature_importance'):
        importance = model.get_feature_importance()
        x_title = 'Contribution Score'
    elif hasattr(model, 'feature_importances_'):
        importance = model.feature_importances_
        x_title = 'Contribution Score'
    else:
        raise ValueError("Model type not supported for importance extraction.")

    df = pd.DataFrame({
        'Feature': columns,
        'Importance': importance
    }).sort_values(by='Importance', ascending=False)

    chart = alt.Chart(df).mark_bar(color=g_color).encode(
        x=alt.X('Importance:Q', title=x_title),
        y=alt.Y('Feature:N', sort='-x', title='Predictor Variable'),
        tooltip=['Feature', 'Importance']
    ).properties(
        title=f'Key Drivers of Energy Demand ({model_name})',
```

```
        width=600,
        height=400
    )

    if save_path:
        chart.save(save_path)

    return chart

## Ridge (Linear Regression)
lin_importance_chart = plot_feature_importance(best_linear_model, 'Ridge Linear', X_train.columns, './figs/lin_imp_chart.png')

## CatBoost
cat_importance_chart = plot_feature_importance(best_cat_model, 'CatBoost', X_train.columns, './figs/xgb_imp_chart.png')

## XGBoost
xgb_importance_chart = plot_feature_importance(best_xgb_model, 'XGBoost', X_train.columns, './figs/cat_imp_chart.png')

display(lin_importance_chart)
display(cat_importance_chart)
display(xgb_importance_chart)


with pd.option_context('display.max_columns', None):
    display(X_train.head())

import pandas as pd
import numpy as np
import altair as alt
import scipy.stats as stats

def plot_qq_grid(models_dict):
    """
    models_dict: {
        'Linear': {'Train': y_p_train, 'Val': y_p_val, 'Test': y_p_test},
        'XGBoost': { ... },
        'CatBoost': { ... }
    }
    """
    all_data = []

    # Nested loop to build the 'Tidy' dataframe
    for model_name, splits in models_dict.items():
        for split_name, y_pred in splits.items():
            # Match the correct true values
            y_true = {'Train': y_train, 'Val': y_val, 'Test': y_test}[split_name]

            # Calculate Standardized Residuals
            res = (y_true - y_pred).values
            res_std = (res - np.mean(res)) / np.std(res)
            res_sorted = np.sort(res_std)

            # Theoretical Normal Quantiles
```

```
            theoretical = stats.norm.ppf(np.linspace(0.01, 0.99, len(res_sorted)))

            tmp_df = pd.DataFrame({
                'Theoretical': theoretical,
                'Sample': res_sorted,
                'Model': model_name,
                'Split': split_name
            })

            # Downsample for browser performance
            if len(tmp_df) > 3000:
                tmp_df = tmp_df.sample(3000, random_state=42).sort_values('
                    ↪ Theoretical')

            all_data.append(tmp_df)

    df = pd.concat(all_data)

    # Plot points
    points = alt.Chart(df).mark_circle(size=20, opacity=0.4).encode(
        x=alt.X('Theoretical:Q', title='Theoretical'),
        y=alt.Y('Sample:Q', title='Residuals'),
        color=alt.Color('Split:N', legend=None)
    ).properties(width=200, height=200)

    # The Reference Line (Standard Normal y=x)
    line_df = pd.DataFrame({'x': [-3, 3], 'y': [-3, 3]})
    line = alt.Chart(line_df).mark_line(color='red', strokeDash=[4,4]).encode(
        x='x:Q', y='y:Q'
    )

    # Combine and facet into 3x3
    grid = alt.layer(points, line, data=df).facet(
        column=alt.Column('Split:N', sort=['Train', 'Val', 'Test']),
        row=alt.Row('Model:N')
    ).resolve_scale(
        x='shared', y='shared'
    )

    return grid

results_grid = {
    'Linear': {'Train': best_linear_model.predict(X_train), 'Val': best_linear_model
        ↪ .predict(X_val), 'Test': best_linear_model.predict(X_test)},
    'XGBoost': {'Train': best_xgb_model.predict(X_train), 'Val': best_xgb_model.
        ↪ predict(X_val), 'Test': best_xgb_model.predict(X_test)},
    'CatBoost': {'Train': best_cat_model.predict(X_train), 'Val': best_cat_model.
        ↪ predict(X_val), 'Test': best_cat_model.predict(X_test)}
}
plot_qq_grid(results_grid)
```